



Nama: **Zefanya Danovanta Tarigan (122140101)**

Tugas Ke: **Tugas Besar Multimedia**

Mata Kuliah: **Sistem Teknologi Multimedia (IF25-40305)**

Tanggal: 11 Desember 2025

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi pengolahan citra dan *computer vision* dalam beberapa tahun terakhir memungkinkan munculnya berbagai aplikasi interaktif yang dapat memodifikasi tampilan wajah pengguna secara waktu nyata (*real-time*). Salah satu implementasi yang paling populer adalah teknologi *Augmented Reality (AR)* yang digunakan pada platform seperti Instagram, TikTok, dan Snapchat, di mana pengguna dapat menambahkan berbagai efek visual berupa topi, kacamata, kumis, ataupun objek digital lainnya yang mengikuti pergerakan wajah secara akurat [1].

Proyek ini dikembangkan sebagai bagian dari penyelesaian tugas besar mata kuliah Sistem Teknologi Multimedia. Sistem yang dibuat bertujuan untuk membangun aplikasi *Party Clown Filter*, yaitu aplikasi berbasis kamera yang mampu mendeteksi wajah dan tangan pengguna untuk kemudian memasang berbagai aksesoris digital seperti topi, kacamata, dan kumis. Seluruh proses berjalan secara *real-time* menggunakan algoritma *face landmarking* dan *hand landmarking* dari library MediaPipe [2].

MediaPipe menyediakan titik landmark wajah yang mampu mendeteksi kepala, sudut rotasi wajah, serta lokasi presisi fitur wajah seperti mata, hidung, dan mulut. Kemampuan ini dimanfaatkan untuk mengatur posisi, ukuran, dan orientasi aksesoris digital agar selalu mengikuti gerakan pengguna secara stabil. Selain itu, *hand tracking* digunakan sebagai mekanisme interaksi berbasis gestur, di mana pengguna dapat memilih atau mengganti aksesoris hanya dengan menyentuh ikon aksesoris pada layar [3].

Sistem kemudian memvisualisasikan hasil akhir melalui antarmuka grafis berbasis Tkinter, dengan proses *overlay* gambar PNG yang dirender menggunakan OpenCV. Pendekatan ini menghasilkan aplikasi yang responsif dan menyerupai cara kerja filter AR pada perangkat modern. Dengan demikian, proyek ini mampu mengintegrasikan antara deteksi wajah, analisis landmark, gestur, dan perancangan antarmuka *real-time* yang interaktif.

2 Alat dan Bahan

Untuk membangun dan menjalankan aplikasi *Party Clown Virtual Accessory Filter*, diperlukan beberapa alat dan bahan yang mendukung proses pengembangan serta eksekusi sistem secara keseluruhan. Adapun rincian alat dan bahan yang digunakan adalah sebagai berikut.

2.1 Perangkat Keras

- **Laptop / Komputer** Digunakan sebagai perangkat utama untuk pengembangan dan eksekusi aplikasi.
- **Webcam Internal atau Eksternal** Berfungsi untuk menangkap video secara *real-time* sebagai input sistem.

2.2 Perangkat Lunak

- **Python 3.11** Bahasa pemrograman utama dalam pembuatan sistem.
- **Visual Studio Code**

2.3 Pustaka Python (*Library*)

Pengembangan aplikasi ini memanfaatkan beberapa pustaka Python sebagai berikut:

- **OpenCV (cv2)** - akses kamera, melakukan berbagai operasi untuk data citra
- **MediaPipe** - untuk deteksi titik landmark wajah dan tangan secara real-time
- **NumPy** - untuk manipulasi data numerik, perhitungan skala aksesori, dan transformasi koordinat.
- **Tkinter** - untuk menampilkan user interface
- **Pillow (PIL)** - untuk mengubah frame OpenCV (BGR) menjadi format gambar yang sesuai dengan Tkinter (RGB).
- **pygame** - untuk memainkan backsound selama aplikasi berjalan.

2.4 Dataset Gambar Aksesori

Aplikasi memerlukan beberapa aset gambar berformat PNG di antaranya:

- Topi musim dingin (*winter hat*)
- Topi wisuda
- Topi pantai
- Topi sihir
- Kacamata 3D, kacamata cinta, kacamata kaya, kacamata 8-bit
- Kumis

Contoh dataset gambar (Aksesoris) yang digunakan :



Gambar 1: Contoh Aksesoris

Seluruh gambar berada dalam direktori **Images/** dan digunakan sebagai objek virtual yang ditempelkan ke wajah pengguna.

2.5 Struktur Folder Pendukung

Struktur folder dalam proyek diatur sebagai berikut:

- **accessories/** – berisi modul aksesoris (topi, kacamata, kumis, overlay).
- **utils/** – berisi utilitas seperti `handutils.py` dan `resizeimg.py`.
- **Images/** – berisi aset PNG yang akan dirender ke wajah.
- **Background/** – berisi file musik yang diputar saat aplikasi berjalan.

2.6 Metode dan Algoritma

Berikut merupakan metode dan algoritma yang digunakan dalam pengembangan proyek Virtual Accessory Filter ini:

1. **Deteksi Landmark Wajah :** Sistem menggunakan algoritma *Face Mesh* dari MediaPipe untuk mendeteksi hingga 468 titik landmark wajah secara real-time. Setiap landmark memiliki koordinat 3D relatif terhadap dimensi frame. Landmark-landmark penting—seperti mata kiri (33), mata kanan (263), dahi (10), dan bibir (13, 14)—digunakan sebagai acuan untuk menentukan posisi aksesoris [4]. Algoritma ini memanfaatkan model regresi dan koneksi topologi mesh wajah untuk menjaga stabilitas prediksi meskipun terjadi rotasi kepala, perubahan ekspresi, atau variasi pencahayaan.
2. **Deteksi Landmark Tangan :** Untuk pemilihan aksesoris, digunakan MediaPipe Hands yang menghasilkan 21 landmark per tangan. Setiap titik koordinat digunakan untuk mendeteksi apakah jari tangan menyentuh area tombol aksesoris di layar [5]. Dengan membandingkan posisi landmark dengan koordinat area ikon, sistem menentukan kapan pengguna “menekan” suatu ikon tanpa memerlukan klik fisik.
3. **Perhitungan Sudut Rotasi Wajah:** Agar topi, kacamata, dan kumis dapat berputar mengikuti orientasi kepala, sudut rotasi dihitung menggunakan dua landmark mata (33 dan 263). Metode ini memungkinkan aksesoris untuk selalu mengikuti arah miring kepala, baik ke kiri maupun ke kanan, sebagaimana lazimnya digunakan pada sistem *face alignment*.
4. **Transformasi Gambar Aksesoris (Rotasi dan Skala):** Setiap aksesoris seperti topi, kacamata, dan kumis dimodifikasi menggunakan operasi transformasi berikut:
 - **Scaling** – ukuran disesuaikan dengan jarak landmark wajah (misalnya jarak antar mata untuk kacamata, lebar mulut untuk kumis).
 - **Rotation** – gambar diputar menggunakan matriks rotasi OpenCV berdasarkan sudut kepala.
 - **Point Rotation** – beberapa fitur wajah seperti dahi diputar mengelilingi titik pivot agar posisi aksesoris benar-benar mengikuti wajah secara presisi.

Pendekatan ini memastikan aksesoris tetap proporsional dan selaras dengan pergerakan kepala pengguna.

3 Penjelasan Program

Program *Party Clown Filter* dibangun menggunakan bahasa pemrograman Python dengan memanfaatkan pustaka OpenCV dan MediaPipe untuk melakukan deteksi wajah dan tangan secara real-time. Aplikasi ini menambahkan berbagai aksesoris virtual (topi, kacamata, dan kumis) pada wajah pengguna berdasarkan hasil deteksi landmark wajah.

Berikut adalah penjelasan masing-masing modul dan fungsinya.

3.1 Modul main.py

Modul `main.py` berfungsi sebagai *entry point* aplikasi. Modul ini membangun menu utama berbasis Tkinter yang menampilkan judul aplikasi dan sebuah tombol untuk memulai kamera. Ketika tombol `START CAMERA` ditekan, modul ini:

- Memutar backsound menggunakan fungsi `play_backsound()` dari `backsound.py`.
- Menutup jendela menu utama.
- Membuka jendela baru yang menjalankan kelas `CameraGUI` dari `gui.py`.

```

1 import tkinter as tk
2 from gui import CameraGUI
3 from backsound import play_backsound
4
5 #MAIN PROGRAM
6
7 class MainApp:
8     def __init__(self, root):
9         self.root = root
10        self.root.title("Menu Utama")
11        self.root.geometry("500x350")
12        self.root.configure(bg="#1e1e1e")
13
14        # NAMA TUBES
15        title = tk.Label(
16            root,
17            text="Party Clown Filter",
18            font=("Poppins", 24, "bold"),
19            fg="white",
20            bg="#1e1e1e"
21        )
22        title.pack(pady=40)
23
24        # Tombol Start
25        self.start_btn = tk.Button(
26            root,
27            text="START CAMERA",
28            font=("Poppins", 16, "bold"),
29            bg="#4CAF50",
30            fg="white",
31            activebackground="#45a049",
32            activeforeground="white",
33            relief="flat",
34            width=15,
35            height=2,
36            command=self.open_camera
37        )
38        self.start_btn.pack(pady=20)
39
40        # Hover Effect
41        self.start_btn.bind("<Enter>", lambda e: self.start_btn.config(bg="#58d164"))
42        self.start_btn.bind("<Leave>", lambda e: self.start_btn.config(bg="#4CAF50"))
43
44        def open_camera(self):
45            # Play Backsound
46            play_backsound("Backsound/game-8-bit.mp3")
47
48            # Tutup menu utama
49            self.root.destroy()
50

```

```

51     # Masuk kamera
52     cam_window = tk.Tk()
53     CameraGUI(cam_window)
54     cam_window.mainloop()
55
56 if __name__ == "__main__":
57     root = tk.Tk()
58     app = MainApp(root)
59     root.mainloop()

```

Kode 1: main.py - Entry Point dan Menu Utama

3.2 Modul gui.py

Modul gui.py merupakan inti dari aplikasi. Modul ini untuk membangun antarmuka kamera, menangani input pengguna, dan menerapkan aksesoris virtual ke wajah pengguna secara real-time.

3.2.1 library

Kode ini memuat seluruh import library yang digunakan oleh GUI, termasuk OpenCV, Tkinter, MediaPipe, dan modul internal seperti aksesoris dan utilitas. Pemisahan ini membantu menjaga cameraGUI.py tetap bersih dari deretan impor yang panjang.

```

1 import cv2
2 from tkinter import Label, Button
3 from PIL import Image, ImageTk
4 import os
5 from camera import Camera
6 import time
7 from landmark import FaceLandmark, HandLandmark
8 from backsound import stop_backsound
9 from accessories.overlay import overlay_png
10 from accessories.transform import get_face_angle
11 from accessories.hat import render_hat
12 from accessories.glasses import render_glasses
13 from accessories.mustache import render_mustache
14 from utils.resizeimg import resize_with_aspect_ratio
15 from utils.handutils import hand_touching_rect

```

Kode 2: Import Library

3.2.2 Class cameraGUI

File ini berisi definisi kelas CameraGUI, yang merupakan pusat logika aplikasi. Tanggung jawab utama kelas ini meliputi:

1. Menyiapkan jendela Tkinter dalam mode *fullscreen*.
2. Menginisialisasi kamera, detektor landmark wajah, dan detektor tangan.
3. Memuat ikon aksesoris beserta posisi dan kategorinya.
4. Menangani pemilihan aksesoris melalui deteksi sentuhan tangan pada ikon.
5. Mengatur pemanggilan fungsi `update_frame()` secara periodik menggunakan `root.after()`.

```

1 class CameraGUI:
2     def __init__(self, root):
3         self.root = root
4         self.root.title("Live Camera")
5
6         # landmark detektor
7         self.face = FaceLandmark()
8         self.hand = HandLandmark()
9
10        self.root.attributes("-fullscreen", True)
11        self.root.bind("<Escape>", lambda e: self.root.attributes("-fullscreen", False))
12
13        # kamera
14        try:
15            self.cam = Camera(0)
16        except:
17            self.cam = Camera(1)
18
19        self.last_touch_time = 0
20        self.touch_delay = 0.5
21        accessories_info = [
22            ("winter", os.path.join(IMG_DIR, "winter-hat.png"), "hat"),
23            ("gradu", os.path.join(IMG_DIR, "gradu-hat.png"), "hat"),
24            ("beach", os.path.join(IMG_DIR, "beach-hat.png"), "hat"),
25            ("witch", os.path.join(IMG_DIR, "witch-hat.png"), "hat"),
26
27            ("rich", os.path.join(IMG_DIR, "rich-sunglasses.png"), "glasses"),
28            ("love", os.path.join(IMG_DIR, "love-sunglasses.png"), "glasses"),
29            ("3D", os.path.join(IMG_DIR, "3d-sunglasses.png"), "glasses"),
30            ("8bit", os.path.join(IMG_DIR, "8bit-sunglasses.png"), "glasses"),
31
32            ("kumis", os.path.join(IMG_DIR, "kumis.png"), "kumis")
33        ]
34
35        self.accessories = {}
36        start_x = 10
37        start_y = 10
38        spacing = 70 # jarak antar icon
39
40        for i, (key, path, kind) in enumerate(accessories_info):
41            img = cv2.imread(path, cv2.IMREAD_UNCHANGED) # BGRA
42            if img is None:
43                raise FileNotFoundError(f"Gambar aksesoris tidak ditemukan: {path}")
44
45            preview = cv2.resize(img, (65, 65), interpolation=cv2.INTER_AREA)
46            pos = (start_x + i * spacing, start_y) # posisi icon di pojok
47
48            self.accessories[key] = {
49                "img": img, # gambar full
50                "preview": preview, # icon kecil
51                "pos": pos, # posisi icon
52                "kind": kind, # nama accessories
53            }
54
55            # Tidak ada topi yang aktif di awal
56            self.active_accessories = set()
57
58            # Area Video
59            self.video_label = Label(self.root)
60            self.video_label.pack()
61
62            # Tombol STOP

```

```

63         self.stop_btn = Button(
64             self.root,
65             text="STOP",
66             font=("Arial", 12, "bold"),
67             bg="red",
68             fg="white",
69             relief="flat",
70             command=self.stop_camera
71         )
72         self.stop_btn.pack(pady=10)
73
74         self.update_frame()
75         self.root.protocol("WM_DELETE_WINDOW", self.stop_camera)

```

Kode 3: Class CameraGUI

3.2.3 Fungsi update_frame

Fungsi `update_frame()` adalah inti proses real-time dalam aplikasi. Fungsi ini dipanggil secara berulang dan melakukan operasi berikut:

1. Mengambil frame terbaru dari kamera dan melakukan mirror flip.
2. Menjalankan deteksi landmark wajah dan tangan menggunakan MediaPipe.
3. Menghitung sudut rotasi wajah melalui `get_face_angle()` dari modul `accessories/transform.py`.
4. Menampilkan ikon aksesoris di bagian atas tampilan.
5. Menggunakan `hand_touching_rect()` untuk menentukan apakah pengguna menyentuh salah satu ikon aksesoris.
6. Menampilkan aksesoris aktif (topi, kacamata, kumis) dengan memanggil:
 - `render_hat()`
 - `render_glasses()`
 - `render_mustache()`
7. Menyesuaikan ukuran frame ke ukuran jendela melalui `resize_with_aspect_ratio()`.
8. Menampilkan frame yang sudah diproses ke antarmuka Tkinter.

```

1 def update_frame(self):
2     frame = self.cam.get_frame()
3     if frame is None:
4         self.root.after(10, self.update_frame)
5         return
6
7     frame = cv2.flip(frame, 1) # mirror effect
8
9     # Deteksi landmark
10    frame, face_points = self.face.process(frame)
11    frame, hand_points = self.hand.process(frame)
12
13    face_angle = 0
14    if 33 in face_points and 263 in face_points:
15        (x1, y1) = face_points[33]
16        (x2, y2) = face_points[263]
17        face_angle = get_face_angle((x1, y1), (x2, y2))

```

```

18
19     for key, info in self.accessories.items():
20         px, py = info["pos"]
21         frame = overlay_png(frame, info["preview"], px, py)
22
23     for key, info in self.accessories.items():
24         px, py = info["pos"]
25         ph, pw = info["preview"].shape[:2]
26         current_time = time.time()
27
28         if hand_touching_rect(hand_points, px, py, pw, ph):
29             if current_time - self.last_touch_time > self.touch_delay:
30                 kind = info["kind"]
31
32                 if key in self.active_accessories:
33                     self.active_accessories.remove(key)
34
35                 else:
36                     to_remove = [
37                         k for k in self.active_accessories
38                         if self.accessories[k]["kind"] == kind
39                     ]
40                     for k in to_remove:
41                         self.active_accessories.remove(k)
42                         self.active_accessories.add(key)
43
44                     self.last_touch_time = current_time
45                 break
46
47     h, w, _ = frame.shape
48
49     # Render aksesori
50     for acc_key in self.active_accessories:
51         info = self.accessories[acc_key]
52         acc_img = info["img"]
53         kind = info["kind"]
54
55         if kind == "hat" and 10 in face_points and 33 in face_points and 263 in face_points:
56             frame = render_hat(frame, acc_img, face_points, face_angle)
57
58         elif kind == "glasses" and 33 in face_points and 263 in face_points:
59             frame = render_glasses(frame, acc_img, face_points, face_angle)
60
61         elif kind == "kumis" and 13 in face_points and 14 in face_points and 61 in face_points and
291 in face_points:
62             frame = render_mustache(frame, acc_img, face_points, face_angle)
63
64     win_w = self.root.winfo_width()
65     win_h = self.root.winfo_height() - 70 # space untuk tombol STOP
66     if win_w > 0 and win_h > 0:
67         frame = resize_with_aspect_ratio(frame, win_w, win_h)
68
69     # Konversi frame BGR      RGB      ImageTk
70     img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
71     img = Image.fromarray(img)
72     imgtk = ImageTk.PhotoImage(image=img)
73     self.video_label.imgtk = imgtk
74     self.video_label.configure(image=imgtk)
75
76     self.root.after(10, self.update_frame)

```

Kode 4: Proses Pembaruan Frame Kamera

3.2.4 Fungsi stop_camera

Fungsi stop_camera() yang menangani penghentian aplikasi. Fungsi ini akan:

- Menghentikan backsound melalui stop_backsound().
- Melepaskan perangkat kamera dengan Camera.release().
- Menutup jendela Tkinter secara aman.

```
1 def stop_camera(self):
2     """Hentikan backsound, matikan kamera, dan tutup window."""
3     stop_backsound()
4     self.cam.release()
5     self.root.destroy()
```

Kode 5: Fungsi Penghentian Kamera

3.3 Modul camera.py

Modul ini menangani akses kamera menggunakan OpenCV.

- Membuka kamera menggunakan cv2.VideoCapture.
- Mengambil frame dari kamera.
- Melepaskan kamera ketika program selesai.

```
1 import cv2
2
3 class Camera:
4     def __init__(self, index=0):
5         # Membuka kamera
6         self.cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)
7
8         if not self.cap.isOpened():
9             raise ValueError("Tidak bisa membuka kamera. Coba Devie lain.")
10
11     def get_frame(self):
12         """Mengambil satu frame dari kamera"""
13         ret, frame = self.cap.read()
14         if not ret:
15             return None
16         return frame
17
18     def release(self):
19         """Menutup kamera"""
20         if self.cap.isOpened():
21             self.cap.release() # # Tutup kamera
```

Kode 6: camera.py - Akses Kamera

3.4 Modul landmark.py

Modul ini menggunakan MediaPipe untuk mendeteksi landmark wajah dan tangan. Dua kelas utama digunakan:

- FaceLandmark – menghasilkan 468 titik wajah.
- HandLandmark – menghasilkan 21 titik tangan.

```

1 import cv2
2 import mediapipe as mp
3
4 class FaceLandmark:
5     """
6     Deteksi Landmark Wajah pada Frame
7     """
8     def __init__(self):
9         self.mp_face = mp.solutions.face_mesh
10        self.face_mesh = self.mp_face.FaceMesh(
11            max_num_faces=1,          # deteksi 1 wajah
12            refine_landmarks=True,
13            min_detection_confidence=0.5,
14            min_tracking_confidence=0.5
15        )
16
17    def process(self, frame):
18        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
19        result = self.face_mesh.process(rgb)
20
21        face_points = {} # {index: (x, y)}
22
23        if result.multi_face_landmarks:
24            h, w, _ = frame.shape
25            for face_landmarks in result.multi_face_landmarks:
26                for idx, lm in enumerate(face_landmarks.landmark):
27                    x = int(lm.x * w)
28                    y = int(lm.y * h)
29                    face_points[idx] = (x, y)
30
31        return frame, face_points
32
33 class HandLandmark:
34     """
35     Deteksi Landmark Tangan pada Frame
36     """
37     def __init__(self):
38         self.mp_hands = mp.solutions.hands
39         self.hands = self.mp_hands.Hands(
40             max_num_hands=2,          # deteksi 2 tangan
41             min_detection_confidence=0.5,
42             min_tracking_confidence=0.5
43         )
44
45    def process(self, frame):
46        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
47        result = self.hands.process(rgb)
48
49        hand_points = [] # list of (x, y) semua titik tangan
50
51        if result.multi_hand_landmarks:
52            h, w, _ = frame.shape
53            for hand in result.multi_hand_landmarks:
54                for lm in hand.landmark:
55                    x = int(lm.x * w)
56                    y = int(lm.y * h)
57                    hand_points.append((x, y))
58        return frame, hand_points

```

Kode 7: landmark.py - Face dan Hand Landmark Detection

3.5 Modul `backsound.py`

Modul ini mengatur pemutaran musik latar menggunakan `pygame.mixer`.

```

1 import pygame
2
3 def play_backsound(path):
4     """
5     play background
6     """
7     pygame.mixer.init()
8     pygame.mixer.music.load(path)
9     pygame.mixer.music.play(-1)
10
11 def stop_backsound():
12     """
13     stop audio
14     """
15     pygame.mixer.music.stop()

```

Kode 8: `backsound.py` - Pengaturan Backsound

3.6 Folder `accessories`

Folder ini berisi seluruh logika pemrosesan gambar aksesoris. Setiap aksesoris memiliki file tersendiri.

3.6.1 `overlay.py`

Mengatur proses *alpha blending* sehingga gambar PNG dengan transparansi dapat digabungkan dengan frame kamera.

```

1 import cv2
2
3 def overlay_png(bg, fg, x, y):
4     """
5     Overlay gambar PNG (bg: BGR, fg: BGRA) ke posisi (x, y) di bg.
6     x, y = posisi kiri-atas fg di bg.
7     """
8     bh, bw = bg.shape[:2]
9     fh, fw = fg.shape[:2]
10
11     # Cek batas biar gak keluar frame
12     if x >= bw or y >= bh:
13         return bg
14     if x + fw <= 0 or y + fh <= 0:
15         return bg
16
17     # Clipping ROI
18     x1 = max(x, 0)
19     y1 = max(y, 0)
20     x2 = min(x + fw, bw)
21     y2 = min(y + fh, bh)
22
23     fg_x1 = x1 - x
24     fg_y1 = y1 - y
25     fg_x2 = fg_x1 + (x2 - x1)
26     fg_y2 = fg_y1 + (y2 - y1)
27
28     roi = bg[y1:y2, x1:x2]
29     fg_roi = fg[fg_y1:fg_y2, fg_x1:fg_x2]
30
31     if fg_roi.shape[2] == 4:

```

```

32     b, g, r, a = cv2.split(fg_roi)
33     alpha = a.astype(float) / 255.0
34     alpha = alpha[..., None]
35
36     fg_rgb = cv2.merge((b, g, r)).astype(float)
37     bg_rgb = roi.astype(float)
38
39     blended = alpha * fg_rgb + (1 - alpha) * bg_rgb
40     bg[y1:y2, x1:x2] = blended.astype("uint8")
41
42 else:
43     bg[y1:y2, x1:x2] = fg_roi[:, :, :3]
44
45 return bg

```

Kode 9: overlay.py - Alpha Blending

3.6.2 transform.py

Mengatur transformasi gambar seperti rotasi PNG dan rotasi titik berdasarkan sudut kepala.

```

1 import cv2
2 import math
3
4 def get_face_angle(p1, p2):
5     """
6     Menghitung sudut rotasi wajah berdasarkan dua titik landmark.
7     p1 = titik landmark pertama (x1, y1)
8     p2 = titik landmark kedua (x2, y2)
9     """
10
11     dx = p2[0] - p1[0]
12     dy = p2[1] - p1[1]
13     angle = math.degrees(math.atan2(dy, dx))
14     return angle
15
16 def rotate_png(png, angle):
17     """
18     Merotasi gambar PNG (dengan transparansi) dengan sudut tertentu.
19     png = gambar (BGRA)
20     angle = sudut rotasi dalam derajat (positif = searah jarum jam)
21     """
22
23     h, w = png.shape[:2]
24     center = (w // 2, h // 2)
25
26     M = cv2.getRotationMatrix2D(center, angle, 1.0)
27
28     # rotasi PNG dengan warpAffine
29     rotated = cv2.warpAffine(
30         png, M, (w, h),
31         flags=cv2.INTER_LINEAR,
32         borderMode=cv2.BORDER_CONSTANT,
33         borderValue=(0, 0, 0, 0)
34     )
35     return rotated
36
37 def rotate_point(cx, cy, px, py, angle_deg):
38     """
39     rotasi titik (px, py) pada pusat rotasi (cx, cy) sebesar angle_deg.
40     """
41

```

```

42 angle = math.radians(angle_deg)
43 s = math.sin(angle)
44 c = math.cos(angle)
45
46 px -= cx
47 py -= cy
48
49 xnew = px * c - py * s
50 ynew = px * s + py * c
51
52 px = xnew + cx
53 py = ynew + cy
54 return int(px), int(py)

```

Kode 10: transform.py - Transformasi Aksesori

3.6.3 hat.py

Mengatur penempatan dan orientasi topi berdasarkan landmark dahi dan mata.

```

1 import math
2 import cv2
3 from accessories.transform import rotate_png, rotate_point
4 from accessories.overlay import overlay_png
5
6 def render_hat(frame, acc_img, face_points, face_angle):
7     "Fungsi untuk menampilkan dan mengatur posisi kacamata"
8
9     HAT_SCALE = 2.3      # mengatur ukuran topi
10    HAT_OFFSET_Y = 13     # mengatur tinggi topi (semakin negatif semakin naik)
11    ho, wo = acc_img.shape[:2]
12
13    # pivot rotasi = midpoint antara mata kanan & mata kiri (landmark 33 & 263)
14    (x1, y1) = face_points[33]
15    (x2, y2) = face_points[263]
16    pivot_x = (x1 + x2) // 2
17    pivot_y = (y1 + y2) // 2
18
19    # titik dahi (landmark 10)
20    fx, fy = face_points[10]
21
22    # ukuran topi mengikuti lebar dahi
23    face_width = math.hypot(x2 - x1, y2 - y1)
24
25    hat_w = int(face_width * HAT_SCALE)
26    hat_h = int(hat_w * ho / wo)
27
28    # Resize gambar topi sesuai dahi
29    hat_img = cv2.resize(acc_img, (hat_w, hat_h))
30
31    # Rotasi topi dari kemiringan kepala
32    hat_img = rotate_png(hat_img, -face_angle)
33
34    # dahi ikut diputar agar topi benar-benar mengikuti arah kepala
35    rotated_fx, rotated_fy = rotate_point(pivot_x, pivot_y, fx, fy, face_angle)
36
37    # posisi final topi
38    hat_x = rotated_fx - hat_w // 2
39    hat_y = rotated_fy - hat_h + HAT_OFFSET_Y
40
41    frame = overlay_png(frame, hat_img, hat_x, hat_y)

```

```
42 return frame
```

Kode 11: hat.py - Render Topi

3.6.4 glasses.py

Mengatur ukuran dan rotasi kacamata berdasarkan jarak kedua mata.

```
1 import math
2 import cv2
3 from accessories.transform import rotate_png
4 from accessories.overlay import overlay_png
5
6 def render_glasses(frame, acc_img, face_points, face_angle):
7     "Fungsi untuk menampilkan dan mengatur posisi kacamata"
8     # tinggi (ho) lebar (wo) dari kacamata (PNG)
9     ho, wo = acc_img.shape[:2]
10
11     # 33 = mata kanan, 263 = mata kiri
12     (x1, y1) = face_points[33]
13     (x2, y2) = face_points[263]
14
15     # lebar wajah dari jarak kedua mata
16     width = math.hypot(x2 - x1, y2 - y1)
17     gw = int(width * 1.8)
18     gh = int(gw * ho / wo)
19
20     # Resize gambar kacamata sesuai wajah
21     glass = cv2.resize(acc_img, (gw, gh))
22
23     # Rotasi kacamata dari kemiringan kepala
24     glass = rotate_png(glass, -face_angle)
25
26     # pusat lokasi kacamata
27     cx = int((x1 + x2) / 2)
28     cy = int((y1 + y2) / 2)
29
30     frame = overlay_png(frame, glass, cx - gw // 2, cy - gh // 2)
31     return frame
```

Kode 12: glasses.py - Render Kacamata

3.6.5 mustache.py

Mengatur posisi kumis berdasarkan koordinat bibir dan rotasi wajah.

```
1 import math
2 import cv2
3 from accessories.transform import rotate_png, rotate_point
4 from accessories.overlay import overlay_png
5
6 def render_mustache(frame, acc_img, face_points, face_angle):
7     "Fungsi untuk menampilkan dan mengatur posisi kumis"
8
9     MUSTACHE_Y_OFFSET = -10
10    ho, wo = acc_img.shape[:2]
11
12    # center bibir
13    (ux, uy) = face_points[13]
14    (lx, ly) = face_points[14]
15    mx = (ux + lx) // 2
16    my = (uy + ly) // 2
```

```

17
18 # mulut kiri & kanan
19 (ml_x, ml_y) = face_points[61]
20 (mr_x, mr_y) = face_points[291]
21
22 mouth_width = math.hypot(mr_x - ml_x, mr_y - ml_y)
23
24 # scaling
25 mw = int(mouth_width * 1.3) # diperbesar
26 mh = int(mw * ho / wo)
27
28 mustache = cv2.resize(acc_img, (mw, mh))
29 mustache = rotate_png(mustache, -face_angle)
30
31 # rotasi pusat mulut
32 cx = (ml_x + mr_x) // 2
33 cy = (ml_y + mr_y) // 2
34
35 mx2, my2 = rotate_point(cx, cy, mx, my, face_angle)
36
37 frame = overlay_png(frame, mustache, mx2 - mw // 2, my2 - mh // 2 + MUSTACHE_Y_OFFSET)
38 return frame

```

Kode 13: mustache.py - Render Kumis

3.7 Folder utils

3.7.1 resizeimg.py

Mengatur fungsi *resize with aspect ratio* agar tampilan kamera tidak terdistorsi.

```

1 import cv2
2 import numpy as np
3
4 def resize_with_aspect_ratio(image, target_w, target_h):
5     #ukuran asli
6     h, w = image.shape[:2]
7     scale = min(target_w / w, target_h / h)
8
9     new_w = int(w * scale)
10    new_h = int(h * scale)
11
12    # Resize gambar dengan ukuran baru
13    resized = cv2.resize(image, (new_w, new_h), interpolation=cv2.INTER_AREA)
14
15    canvas = np.zeros((target_h, target_w, 3), dtype=np.uint8)
16    x_offset = (target_w - new_w) // 2
17    y_offset = (target_h - new_h) // 2
18
19    canvas[y_offset:y_offset+new_h, x_offset:x_offset+new_w] = resized
20    return canvas

```

Kode 14: resizeimg.py - Resize Frame

3.7.2 handutils.py

Berisi fungsi untuk mendeteksi apakah tangan menyentuh area ikon aksesoris.

```

1 def hand_touching_rect(hand_points, x, y, w, h):
2     """
3     cek titik-titik tangan (hasil landmark Mediapipe)
4     menyentuh atau berada di dalam area persegi (rectangle) tertentu.

```

```

5
6     hand_points : list berisi koordinat (px, py) dari landmark tangan
7     x, y         : posisi kiri-atas rectangle
8     w, h         : lebar dan tinggi rectangle
9     """
10
11     if not hand_points: # None atau list kosong
12         return False # tidak ada yang menyentuh
13
14     for px, py in hand_points:
15         if x <= px <= x + w and y <= py <= y + h:
16             return True # ada titik tangan masuk ke area rect
17
18     return False

```

Kode 15: handutils.py - Deteksi Sentuhan Tangan

4 Instalasi Program

Sebelum menjalankan aplikasi *Party Clown Virtual Accessory Filter*, pastikan komputer telah memenuhi prasyarat teknis berikut.

4.1 Prasyarat Sistem

- **Python Version:** Aplikasi ini direkomendasikan untuk dijalankan pada Python versi 3.10 atau 3.11 agar seluruh pustaka seperti OpenCV, MediaPipe, dan Pygame dapat berfungsi tanpa masalah kompatibilitas.
- **Webcam:** Diperlukan kamera internal atau eksternal yang mendukung streaming video untuk memastikan proses overlay aksesoris dapat berjalan secara real-time.
- **Sistem Operasi:** Aplikasi didukung pada Windows 10/11, macOS, dan Linux.
- **Dependencies Manager:** Pastikan **pip** sudah terinstal sebagai manajer paket Python.

4.2 Clone Repository Proyek

Langkah pertama untuk memulai instalasi adalah memperoleh kode sumber aplikasi. Jika proyek disimpan pada GitHub atau Git hosting lainnya, gunakan perintah berikut untuk mengunduh repository:

```

1 git clone https://github.com/danovantaa/PartyClown-VirtualAccessoryFilter.git
2 cd PartyClown-VirtualAccessoryFilter

```

Pastikan **git** telah terinstal pada perangkat Anda. Jika belum, instal melalui *Git for Windows* atau package manager pada macOS/Linux.

4.3 Membuat dan Mengaktifkan Virtual Environment

Untuk menjaga dependensi tetap terisolasi dari sistem utama, buat lingkungan virtual menggunakan perintah berikut:

```

1 python -m venv venv

```

Aktifkan virtual environment:

- **Windows:**

```

1 venv\Scripts\activate

```


- **macOS/Linux:**

```
1 source venv/bin/activate
```

4.4 Instalasi Dependensi

Aplikasi membutuhkan beberapa pustaka utama seperti OpenCV, MediaPipe, NumPy, Pillow, dan pygame. Semua dependensi ini telah didefinisikan dalam file `requirements.txt`. Jalankan perintah berikut untuk menginstalnya:

```
1 pip install -r requirements.txt
```

4.5 Menjalankan Aplikasi

Setelah seluruh dependensi berhasil terpasang, aplikasi dapat dijalankan langsung melalui `main.py`. Pastikan Anda berada di dalam direktori proyek, lalu jalankan:

```
1 python main.py
```

Ketika dijalankan:

- Aplikasi akan membuka jendela menu utama.
- Tombol **START CAMERA** akan memulai kamera dan menjalankan modul `CameraGUI`.
- Musik latar (background) akan diputar otomatis melalui modul `pygame`.
- Pengguna dapat memilih aksesoris dengan menyentuh ikon aksesoris menggunakan tangan (gesture-based selection).

Jika kamera tidak terdeteksi pada indeks 0, program otomatis beralih ke indeks 1 sesuai implementasi pada kelas `Camera`.

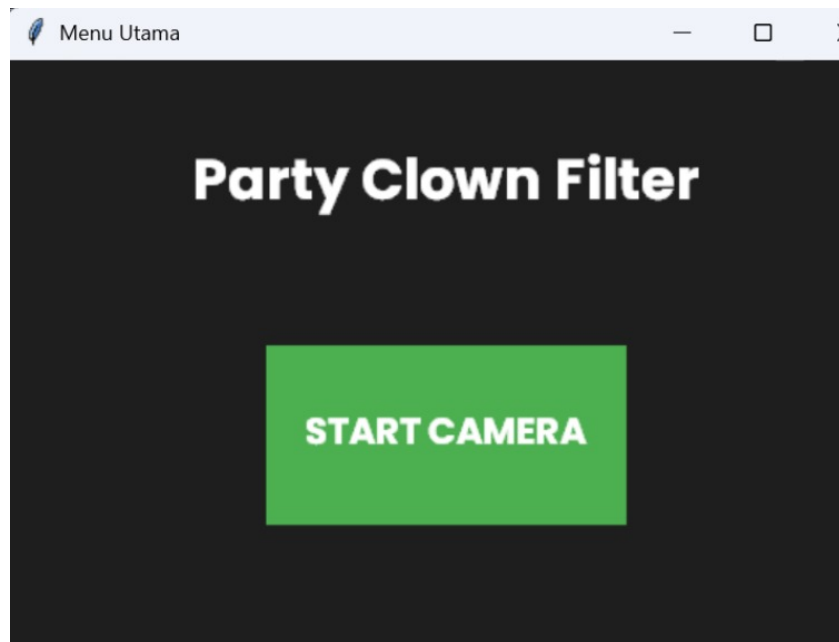
5 Hasil dan Pembahasan

5.1 Desain Eksperimen

Pengujian dilakukan untuk mengevaluasi kinerja sistem *Party Clown Virtual Accessory Filter* dalam mendeteksi wajah, mendeteksi tangan, dan menerapkan berbagai aksesoris virtual seperti topi, kacamata, dan kumis secara *real-time*. Pengujian dilakukan menggunakan webcam.

Pengguna diminta untuk menghadap ke kamera dan mencoba mengaktifkan aksesoris dengan “menyentuh” ikon aksesoris yang muncul di bagian atas layar menggunakan tangan. MediaPipe FaceMesh digunakan untuk melacak landmark wajah, sedangkan MediaPipe Hands digunakan untuk mendeteksi koordinat tangan yang digunakan sebagai kontrol input pengguna.

5.2 Visualisasi Sistem



Gambar 2: Tampilan Awal GUI



Gambar 3: Tampilan Aplikasi Dengan Aksesori Virtual

Analisis visual:

- Ikon aksesori ditampilkan pada bagian atas frame dan dapat dipilih dengan gerakan tangan.
- Aksesori seperti topi, kacamata, dan kumis mengikuti orientasi wajah menggunakan perhitungan sudut dari pasangan landmark mata (33 dan 263).
- Topi menyesuaikan rotasi kepala, sedangkan kacamata dan kumis mengikuti landmark wajah secara proporsional.
- Sistem mampu mempertahankan frame rate yang stabil, sehingga pengalaman interaktif berjalan mulus.

5.3 Analisis Perilaku Aksesori

Aksesori berhasil mengikuti pergerakan wajah berdasarkan beberapa pengamatan berikut:

- **Topi** mengikuti rotasi wajah dan menyesuaikan posisi dengan landmark dahi (landmark 10), namun presisi rotasi sangat dipengaruhi oleh kualitas deteksi landmark.
- **Kacamata** menyesuaikan ukuran berdasarkan jarak antar mata (landmark 33 dan 263), sehingga tetap proporsional meskipun wajah maju/mundur.
- **Kumis** mengikuti pergerakan bibir atas dan bergerak stabil selama landmark mulut terdeteksi dengan baik.

5.4 Kelebihan dan Keterbatasan

Kelebihan:

- Antarmuka interaktif dengan kontrol berbasis gesture.
- Aksesori mengikuti orientasi wajah secara dinamis.
- Penggunaan MediaPipe FaceMesh menghasilkan landmark yang cukup akurat.
- Sistem berjalan secara *real-time* bahkan pada perangkat dengan spesifikasi menengah.

Keterbatasan:

- Topi terkadang tidak sempurna mengikuti rotasi kepala ekstrem (misalnya kepala menunduk tajam).
- Aksesori hanya terbatas pada objek 2D (gambar PNG), belum mendukung model 3D.

5.5 Potensi Pengembangan

Aplikasi memiliki potensi untuk dikembangkan lebih lanjut, terutama untuk kebutuhan hiburan, filter media sosial, maupun augmented reality. Beberapa potensi pengembangan adalah:

- Meningkatkan presisi rotasi topi menggunakan perhitungan vektor 3D dari landmark FaceMesh.
- Menambahkan aksesori berbasis model 3D agar tampilan lebih realistis.
- Meningkatkan sensitivitas gesture khusus seperti “pinch” atau “wave” untuk mengganti aksesori.
- Menambahkan fitur screenshot atau rekaman video langsung dari aplikasi.

6 Kesimpulan

Berdasarkan implementasi dan pengujian terhadap sistem *Party Clown Virtual Accessory Filter*, dapat disimpulkan bahwa aplikasi ini berhasil menerapkan aksesori virtual seperti topi, kacamata, dan kumis secara *real-time* dengan memanfaatkan deteksi landmark dari MediaPipe FaceMesh dan MediaPipe Hands.

Aplikasi mampu:

- Mendeteksi wajah dan orientasinya untuk menempatkan aksesori secara presisi.
- Menyesuaikan ukuran aksesori berdasarkan proporsi wajah pengguna.

- Menangani interaksi berbasis gesture menggunakan tangan dengan tingkat responsivitas yang baik.
- Menjalankan pemrosesan video dan overlay aksesoris secara stabil dalam lingkungan Tkinter.

Dengan hasil tersebut, sistem berhasil memenuhi tujuan utama yaitu membuat aplikasi filter virtual yang interaktif dengan kontrol gesture serta overlay aksesoris yang mengikuti pergerakan wajah pengguna.

Saran

Untuk pengembangan lebih lanjut, beberapa rekomendasi dapat dipertimbangkan:

- Mengimplementasikan model 3D untuk aksesoris agar tampilan lebih realistis.
- Menambahkan fitur perekaman video, tangkapan layar, dan penyimpanan hasil.

Secara keseluruhan, aplikasi ini dapat dijadikan dasar untuk pengembangan sistem *augmented reality* berbasis komputer visi yang lebih kompleks dan profesional.

References

- [1] M. D. Rani and A. Pramono, “Implementasi filter instagram berbasis augmented reality pada lukisan menggunakan metode markerless dan marker based sebagai media promosi di vin autism gallery,” *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 8, no. 5, pp. 10 743–10 750, 2024.
- [2] A. Vakunov, C.-L. Chang, F. Zhang, G. Sung, M. Grundmann, and V. Bazarevsky, “Mediapipe hands: On-device real-time hand tracking,” in *Workshop on Computer Vision for AR/VR*, vol. 2, no. 4, 2020, p. 5.
- [3] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “Mediapipe hands: On-device real-time hand tracking,” *arXiv preprint arXiv:2006.10214*, 2020.
- [4] I. A. Miharja, “Ta: Sistem pembacaan deretan empat angka secara computer vision melalui deteksi gesture jari tangan menggunakan mediapipe,” Ph.D. dissertation, Universitas Dinamika, 2024.
- [5] A. D. RACHMANTO and Y. I. NURHASANAH, “Implementasi hand tracking pada markerless augmented reality dalam aplikasi pembelajaran angklung,” *e-Proceeding FTI*, 2022.